

When it comes to the architecture, another issue that you may come across is DSP versus general-purpose microcontroller.

"Honestly, the line between the two is narrowing by the day. Microcontrollers are designed for simple math and control (of other devices) functions. Some may also come with hardware multiplication units. DSPs are processors optimised for processing the streaming signal. These have special instructions that speed-up common tasks such as multiply-accumulate in a single instruction, as well as other vector or SIMD (single-instruction multiple-data) instructions. They are usually designed to operate in one big loop, processing a data stream. DSPs can be designed as integer, fixed-point or floating-point processors," adds Garadi.

Development tools

As microcontrollers have been available for a very long time, the selection decision in the industry is now governed by the support from chip vendor and development tools, not the price and features.

Siddharth Patel, lead engineer-embedded software, EATON India Engineering Centre, explains: "Now we have multiple vendors offering chips with the same CPU core and similar features at almost similar prices across various segments. So now chip vendor support and development tools play a very important role among selection verticals."

Sai Ram Mannar, founder and director, GreenOcean Research Labs, adds, "The availability and richness of the development tools is a key aspect. As these tools are expensive and mostly tied to the microcontroller, they need to be evaluated clearly, ensuring flexibility of the tool in supporting multiple families with ease. The debugging capabilities and an easy graphic user interface (GUI) for configuring the various peripherals of the MCU also help a lot."

How much should you focus on application requirements?

The application requirements of an embedded system under development

“**Chip vendor support and development tools play a very important role among selection verticals** — Siddharth Patel, lead engineer-embedded software, EATON India Engineering Centre

TOP 7 Factors to be Considered

1. 8-bit vs 32-bit core. 8-bit cores are currently the most widely used MCU cores. For almost all tasks, an 8-bit core like 8051, PIC and AVR will suffice. However, as the project complexity increases, some may want to utilise the 32-bit cores like ARM, which provide you with more features and faster clock speeds but tend to have a higher learning curve than 8-bit cores. Some 32-bit cores also have floating-point units, which enable you to process math/DSP related functions faster and much more efficiently. However, you should get familiar with 8-bit cores before moving to 32-bit cores.

2. Programming language. This is indirectly related to the MCU but directly to the tool chain used. The most popular programming languages for MCUs are Assembly (ASM) and Embedded C.

Working with ASM is much more time-consuming but gives you great insight into the architecture and actual working of the MCU. However, when the code size efficiency does not matter as much as the time to develop, Embedded C is the way to go. Again, you can write efficient code in 'C' and some projects require you to use 'C' to keep things simple.

For most of the 8051s available in the Indian market, Keil supports the code in ASM and Embedded C. For the PIC series, MPLAB has a CCS plugin (I use this as opposed to Hi-Tech Compiler), which enables you to code a PIC in 'C'. So before you select the chip, make sure a tool chain with the right options is available for development. Do consider the cost of the same as many of them do not have free versions.

3. GPIOs and features. Manufacturers release many MCUs of similar specifications but different features and resolutions. Select a chip that has all the features with the resolution as per your needs. Try to keep external chip interfacing to a minimum. This will help you in your design. Also make sure your MCU has sufficient input/output (I/O) pins for your needs. Try using alternate modes for peripherals—like 4-bit LCD mode—to save some I/O pins.

4. Cost. If it is your first time working with an MCU, there is no need to purchase an expensive MCU. We all have had our share of burnt MCUs, and likely you will too. So keep the cost low initially. Weigh all the points mentioned here (and elsewhere too) and select an appropriately priced MCU that suits your budget. For absolute beginners, I would recommend AT89S series and P89V51RD2

(expensive but has PWMs) from 8051 family, and PIC16F877A (has ADCs too) from PIC family. In general, 8051s tend to be cheaper than PICs, which are cheaper than Atmega's, but PICs have much more functionality.

5. Availability. Select an MCU that will be available for a long time to come. There is no general rule of thumb to know this, but selecting a popular one is your best bet. (The ones mentioned above are most widely used in many universities.) If for some reason you select an MCU which is not available in India, you may have to pay additional shipping charges to get it, which will be costly as well as frustrating.

6. Online support. This is crucial when you're just venturing out and when you move on to 32-bit cores. Selecting an MCU which has good online support will help you with your ideas and solve most of your problems as the experience of other users is available for your reference. Getting to know existing bugs with tool chains or MCU documentation will help you avert the problem in advance and speed up development.

7. Packaging. For most DIY projects, a DIP package is suitable and easy to work with. SMD packages are a bit difficult to work with without proper tools and therefore not recommended for first timers. Once the design is ready, you can always use SMD equivalents to make it more compact.

—Compiled by Frenoy Osburn at EFY's Community of Electronics Pros (www.forum.electronicsforu.com)



PIC16F877A microcontroller



P89V51RD2 microcontroller used in a system